

Instructor Manual: Technical Whiteboarding

Pedagogy, Code Tracing, & Algorithm Analysis in Advanced JavaScript

Course: Advanced JavaScript / CS Pedagogy

Format: Complete Teaching Guide & Curriculum

Version: 2.0 (Comprehensive)

COMPREHENSIVE CHAPTER INDEX

1. Introduction to Technical Whiteboarding
2. Instructor Lesson Plan (45-Minute Class)
3. How Technical Interviews Really Work
4. Drawing Data Structures on the Board
5. The Forward/Backward Method
6. Teaching Students to Think Like Engineers
7. Complete Walkthrough: Contains Duplicate
8. Writing the JavaScript Solution
9. Tracing Code by Hand (Stack, Heap, Scope)
10. Big-O Complexity Analysis
11. Mock Interview Exercises & Group Dynamics
12. Instructor Quick Cheat Sheets
13. Student Practice Problems
14. Complete Answer Key & Detailed Traces
15. Assessment & Grading Rubrics

CHAPTER 1

Introduction to Technical Whiteboarding

Whiteboarding is often feared by students as an intimidating interrogation of memorized algorithms. As an instructor, your first priority is to reframe whiteboarding as a **structured communication framework**. It is not a performance test of flawless syntax; it is a collaborative design exercise used by modern software engineering teams.

💬 INSTRUCTOR SCRIPT ("WHAT TO SAY")

"Welcome everyone! Today we're going to learn technical whiteboarding. If the thought of coding on a dry-erase board makes your hands sweat, I want you to take a deep breath. Whiteboarding isn't about memorizing syntax or being perfect on the first try. It's about making your thought process visible so an interviewer can see how you break down complex problems."

? QUESTIONS TO ASK STUDENTS

- "Why do you think software companies ask candidates to whiteboard instead of just giving a 2-hour coding test?"
- "When you get stuck on a coding problem at home, what is the very first thing you usually do?"

⚠️ COMMON MISCONCEPTION

Students believe they must start writing JavaScript code immediately after the prompt is read. Explain that jumping straight to code without clarifying requirements is the #1 reason candidates fail technical interviews.

📝 TEACHING TIP & TIMING

Suggested Timing: 5 Minutes

Set a relaxed tone. Write a silly syntactic error on the board intentionally (e.g., forgetting a closing brace) and laugh it off to show that minor syntax mistakes on a whiteboard are normal and expected.

CHAPTER 2

Instructor Lesson Plan (45-Minute Class)

This turn-key 45-minute lesson plan balances direct instruction, live modeling on the board, paired student practice, and debriefing.

Time Block	Phase	Instructor Action	Student Activity
00:00 – 00:05	1. Mindset & Framework	Introduce 6-Phase Whiteboarding Framework; demystify interview expectations.	Active listening & Q&A.
00:05 – 00:15	2. Live Instructor Modeling	Solve "Contains Duplicate" on board. Model visual standards (arrays, T-tables, Set heap).	Observe, take notes, answer check-in questions.
00:15 – 00:25	3. Code Tracing Mechanics	Demonstrate paper tracing: stack frames, block scopes, crossing out values in T-table.	Follow along tracing step-by-step.
00:25 – 00:38	4. Paired Student Exercise	Assign "Two Sum (Brute Force vs Hash Map)" or "Valid Anagram" in peer pairs.	Pair 1: Whiteboarder Pair 2: Interviewer
00:38 – 00:45	5. Class Debrief & Rubric Review	Highlight common mistakes observed during pairs; review grading rubric.	Self-assess and share feedback.

INSTRUCTOR SCRIPT

"During our paired exercise, remember: if you are the interviewer, your job is NOT to give away the answer. Your job is to ask guiding questions like 'What is the runtime of that nested loop?' or 'Can you trace that with sample input \$A_1\$?'"

CHAPTER 3

How Technical Interviews Really Work

Help students understand the interviewer's evaluation scorecard. Interviewers are evaluating 4 distinct axes:

1. **Problem Solving & Decomposition:** Can the candidate break a ambiguous prompt into concrete logical steps?
2. **Communication & Tone:** Does the candidate talk out loud while thinking, or go silent for 5 minutes?
3. **Code Quality & Proficiency:** Is the candidate comfortable with JavaScript idioms, data structures, and edge cases?
4. **Verification & Tracing:** Can the candidate systematically verify their own code without relying on a console/debugger?

QUESTIONS TO ASK STUDENTS

- "If you go quiet for 3 minutes during an interview, what is your interviewer thinking?"
- "Is an $O(n^2)$ solution that works better or worse than an unfinished $O(n)$ solution?"

COMMON MISCONCEPTION

Students think interviewers expect the optimal $O(1)$ or $O(n)$ solution immediately. Clarify that interviewers prefer seeing a working brute-force solution first, followed by incremental optimization.

CHAPTER 4

Drawing Data Structures on the Board

Clear visual diagrams prevent mental overload. Standardize how data structures are drawn across your curriculum:

STANDARD DATA STRUCTURE VISUALS

1. Arrays / Lists: Contiguous boxes with indices directly above.

0	1	2	3
10	25	30	42

2. Hash Maps / Objects: Key-Value T-Tables on the Heap.

```
Heap Object @ 0x20  
{ "a": 1, "b": 2 }
```

3. Linked Lists: Node boxes with value & pointer arrow.

```
[ Val: 5 | Next ] → [ Val: 12 | Next ] → null
```

4. Sets: Box containing unique elements on Heap.

```
Set Object @ 0x10  
Set { 1, 3, 2 }
```

INSTRUCTOR SCRIPT

"Notice how I put memory address tags like 0x10 on heap objects. In JavaScript, primitives live on the stack or in scope bindings, but Objects, Arrays, and Sets are stored on the Heap. Drawing pointer arrows or hex addresses makes memory references crystal clear!"

CHAPTER 5

The Forward/Backward Method

When students don't know how to solve a problem, they panic. Teach them the **Forward/Backward Method** to systematically bridge the gap between input \$A_1\$ and output \$B_1\$.

THE FORWARD / BACKWARD BRIDGE

Forward (\$A_1\$ o \$A_2\$): Input Array [1, 3, 2, 3] → Iterate through elements sequentially.

Backward (\$B_1\$ ← \$B_2\$ ← \$B_3\$):

- \$B_1\$ (Goal): Return true.
- \$B_2\$ (Precursor): How do we know to return true? We encountered a number we saw earlier.
- \$B_3\$ (Precursor): How do we know we saw it earlier? We stored previously seen elements in a Set (\$O(1)\$ lookup).

? QUESTIONS TO ASK STUDENTS

- "If our end goal \$B_1\$ is a boolean `true`, what condition immediately triggers that output?"
- "What structure allows us to answer 'Have I seen this before?' in constant time?"

CHAPTER 6

Teaching Students to Think Like Engineers

Software engineering is about trade-offs. Teach students to compare alternative strategies out loud before coding.

Approach	Time Complexity	Space Complexity	Pros & Cons
1. Brute Force (Nested Loops)	$O(n^2)$	$O(1)$	Simple to conceive, zero extra memory; extremely slow for large datasets.
2. Sorting First	$O(n \log n)$	$O(1)$ or $O(n)$	Better time complexity; mutates input array or requires copy space.
3. Hash Set Lookup	$O(n)$	$O(n)$	Optimal linear time; trades memory ($O(n)$ heap space) for speed.

INSTRUCTOR SCRIPT

"Always state the tradeoffs! Say: 'We can solve this in $O(1)$ space using nested loops, but it costs $O(n^2)$ time. Alternatively, we can trade $O(n)$ space in memory using a Set to bring the time down to $O(n)$. In modern engineering, we usually prioritize time over space.'"

CHAPTER 7

Complete Whiteboard Walkthrough (Contains Duplicate)

Here is the exact step-by-step layout you should produce on the whiteboard during live modeling.

COMPLETE WHITEBOARD BOARD LAYOUT

TOP LEFT: Input (\$A_1\$)

0	1	2	3
1	3	2	3

BOTTOM LEFT: Output (\$B_1\$)

true

(Duplicate '3' found at index 1 & index 3)

TOP RIGHT: Pseudocode / Strategy

1. Create empty Set seen.
2. Loop each num in nums:
 - If `seen.has(num)` → return true.
 - Else → `seen.add(num)`.
3. Return false if loop finishes.

CHAPTER 8

Writing the JavaScript Solution

Model writing clean, idiomatic modern JavaScript (ES6+). Avoid deprecated syntax like `var`.

```
function hasDuplicate(nums) {  
  // Phase 1: Initialize lookup structure on Heap  
  const seen = new Set();  
  
  // Phase 2: Linear iteration across input array  
  for (const num of nums) {  
    if (seen.has(num)) {  
      return true; // Duplicate found!  
    }  
    seen.add(num);  
  }  
  
  return false; // All elements unique  
}
```

⚠ COMMON MISCONCEPTION

Students often write `seen.contains(num)` instead of JS standard `seen.has(num)` or use `.push()` on Sets instead of `.add()`. Remind them that JavaScript Sets use `.has()` and `.add()`.

CHAPTER 9

Tracing Code by Hand (Stack, Heap, Scope, T-Tables)

Code tracing is executing code manually on paper or whiteboard. Emphasize that variables updated in place must be **crossed out**, never erased, to maintain an audit trail of execution.

TRACING BOARD & STATE UPDATE TABLE

CALL STACK FRAME: `hasDuplicate(nums)`

BLOCK SCOPE: `for (const num of nums)`

HEAP MEMORY

Address: `0x10`

Variable	Value / Memory Ref
nums	Array Ref → 0x01
seen	Set Ref → 0x10
num	1 3 2 3

Set { 1, 3, 2 }

Step Trace Log:

1. num=1: Set={1}
2. num=3: Set={1,3}
3. num=2: Set={1,3,2}
4. num=3: **Match in Set!**
→ Return true. Pop stack frame [X].

CHAPTER 10

Big-O Complexity Analysis

Teach students how to calculate Big-O systematically from their code trace:

- **Time Complexity $O(n)$:** We loop through the array of n elements at most once. Each `seen.has()` and `seen.add()` operation takes $O(1)$ time on average. Total runtime = $n \text{ times } O(1) = O(n)$.
- **Space Complexity $O(n)$:** In the worst case (no duplicates), the `seen` Set on the heap will store n distinct values. Total extra space = $O(n)$.

BIG-O GROWTH COMPARISON CHEATSHEET

Big-O	Name	Operations for $N=1,000$	Example
$O(1)$	Constant	1 operation	Array index lookup (<code>arr[i]</code>)
$O(\log n)$	Logarithmic	~10 operations	Binary search on sorted array
$O(n)$	Linear	1,000 operations	Single loop / Set lookup algorithm
$O(n \log n)$	Linearithmic	~10,000 operations	Merge sort, Quick sort
$O(n^2)$	Quadratic	1,000,000 operations	Nested loops (Brute force pairs)

CHAPTER 11

Mock Interview Exercises & Group Dynamics

Run 15-minute peer mock interviews in class using the "Driver & Navigator" pattern:

CLASSROOM FACILITATION TIPS

- **Pairing:** Pair students with different skill levels to foster peer teaching.
- **Role Rotation:** Switch roles at the 15-minute mark so every student whiteboards and interviews.
- **Interviewer Duty:** Emphasize that the student interviewer must take notes and ask clarifying questions, not sit silently.

CHAPTER 12

Instructor Quick Cheat Sheets

INSTANT BOARD CHECKLIST FOR INSTRUCTORS

1. Did the student draw $\$A_1$ (Input) and $\$B_1$ (Output) before coding?
2. Is the array drawn with index numbers above each box?
3. Is the Set / Map object drawn on the Heap with a reference tag?
4. Did the student cross out old variable values during tracing instead of erasing?
5. Did the student state both Time AND Space complexity with justification?

CHAPTER 13

Student Practice Problems

Assign these 3 curated problems to reinforce the whiteboarding methodology:

PROBLEM 1: VALID ANAGRAM (EASY)

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

Constraint: Optimize for $O(n)$ time using a Frequency Counter Map.

PROBLEM 2: TWO SUM (MEDIUM)

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Constraint: Solve in $O(n)$ time using a Map storing `{ number: index }`.

PROBLEM 3: FIND MISSING NUMBER (MEDIUM)

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return the only number in the range that is missing from the array.

Constraint: Solve in $O(n)$ time and $O(1)$ extra space using Gauss Sum formula: $\text{Sum} = \frac{n(n+1)}{2}$.

CHAPTER 14

Complete Answer Key & Detailed Traces

Answer Key: Problem 1 — Valid Anagram

```
function isAnagram(s, t) {
  if (s.length !== t.length) return false;
  const counts = {};
  for (const char of s) {
    counts[char] = (counts[char] || 0) + 1;
  }
  for (const char of t) {
    if (!counts[char]) return false;
    counts[char]--;
  }
  return true;
}
```

Complexity: Time: $O(n)$ | Space: $O(1)$ (bounded by 26 English letters).

Answer Key: Problem 2 — Two Sum

```
function twoSum(nums, target) {
  const map = new Map(); // { num: index }
  for (let i = 0; i < nums.length; i++) {
    const diff = target - nums[i];
    if (map.has(diff)) {
      return [map.get(diff), i];
    }
    map.set(nums[i], i);
  }
  return [];
}
```

Complexity: Time: $O(n)$ | Space: $O(n)$.

Answer Key: Problem 3 — Missing Number

```
function missingNumber(nums) {
  const n = nums.length;
  const expectedSum = (n * (n + 1)) / 2;
  const actualSum = nums.reduce((acc, curr) => acc + curr, 0);
  return expectedSum - actualSum;
}
```

Complexity: Time: $O(n)$ | Space: $O(1)$.

CHAPTER 15

Assessment & Grading Rubrics

Use this standardized 20-point rubric to evaluate student whiteboard presentations:

Category	Points	Exemplary Performance Criteria (Full Points)
1. Problem Comprehension	4 pts	Draws clear A_1 input array & B_1 output. Asks 2+ edge-case questions (empty array, negative numbers, nulls).
2. Architectural Strategy	4 pts	

Category	Points	Exemplary Performance Criteria (Full Points)
		Explains Forward/Backward method. Evaluates brute-force vs hash map tradeoffs out loud before coding.
3. JavaScript Implementation	4 pts	Writes clean ES6+ code. Uses correct methods (<code>Set.prototype.has</code> , <code>Map.prototype.set</code>). Modular and readable.
4. Code Tracing Mechanics	4 pts	Constructs stack frame, block scopes, and heap references. Updates variable T-tables by crossing out values systematically.
5. Complexity & Communication	4 pts	Accurately states Big-O Time & Space complexity. Maintains clear verbal dialogue throughout entire exercise.

INSTRUCTOR CLOSING SUMMARY

"By holding students to these visual standards and systematic tracing habits, you are equipping them with lifelong engineering habits that will set them apart in technical interviews and professional software design!"